

# Nexus Ciel — V3 : Spécification d'Implémentation (build-ready, interne d'abord)

## Nexus Ciel — V3

### Spécification d'Implémentation (build-ready, interne d'abord)

**Statut du document** : V3.0 — spécification d'implémentation destinée à être verrouillée.

**Relation avec V2.5.2** : V2.5.2 est la spécification d'**architecture** (ce qu'EST Nexus, comment il pense, pourquoi les modules existent). Cette V3 est la spécification d'**implémentation** (comment on le CONSTRUIT). La V3 ne modifie NI la vision, NI Manas, NI Belzébuth, NI Great Sage. Elle les rend constructibles.

**Contrainte stratégique** : Nexus Ciel reste **interne d'abord**. Chaque choix ci-dessous privilégie la simplicité opérationnelle sur un hôte maîtrisé, avant toute ambition de distribution.

**Principe directeur** : *Chaque phase produit un système fonctionnel et testable*. Jamais une phase = 30 TODO. À chaque instant, Nexus existe déjà et devient meilleur.

## Chapitre 0 — Comment lire cette V3

Ce document est organisé en trois blocs :

- Les décisions figées** (chapitre 1 + le chapitre ADR final) : ce qu'on ne rediscute plus.
- Les contrats techniques** (chapitres 2 à 12) : stack, déploiement, données, bus, APIs, fournisseurs, sandboxes, valeurs par défaut, JSON versionnés, rapports, sécurité.
- Le plan de construction** (chapitre 13) : phases successives, chacune livrant un Nexus fonctionnel et testable, avec critères PASS/FAIL.

Règle de lecture : si un chapitre technique et l'architecture V2.5.2 semblent se contredire, **V2.5.2 gagne sur le "quoi/pourquoi", la V3 gagne sur le "comment"**. Aucune décision de V3 ne peut altérer la cognition décrite en V2.5.2.

## Chapitre 1 — Matrice des décisions figées (résumé exécutif)

| # | Sujet   | Décision figée V1 interne                  | Reporté (V1.5+)                            |
|---|---------|--------------------------------------------|--------------------------------------------|
| 1 | Langage | <b>Python 3.12</b> , async natif (asyncio) | Extensions Rust ciblées pour les hot paths |
| 2 | API     | <b>FastAPI</b> + Uvicorn                   | Gateway / auth avancée                     |

|    |                                       |                                                                                           |                                                 |
|----|---------------------------------------|-------------------------------------------------------------------------------------------|-------------------------------------------------|
| 3  | Déploiement                           | <b>Docker Compose sur hôte unique maîtrisé</b>                                            | Kubernetes, multi-nœuds                         |
| 4  | État relationnel                      | <b>PostgreSQL 16</b>                                                                      | Sharding                                        |
| 5  | Vecteurs / mémoire sémantique         | <b>pgvector</b> (dans le même PostgreSQL)                                                 | Chroma / FAISS dédié si volumétrie l'exige      |
| 6  | Graphe de savoir-faire                | <b>Tables relationnelles PostgreSQL</b> (adjacence)                                       | Neo4j si le raisonnement graphe devient central |
| 7  | Event Bus                             | <b>Async in-process (Phase 1) → Redis Streams (Phase 2+)</b>                              | NATS / Kafka                                    |
| 8  | Cache sémantique (niveau 1 du router) | <b>Redis</b>                                                                              | —                                               |
| 9  | Isolation Execution Fabric            | <b>3 niveaux : in-process / conteneur / conteneur sans réseau</b>                         | microVM (Firecracker)                           |
| 10 | Sandbox Capability Creation           | <b>Conteneur éphémère, sans réseau, FS temporaire, rollback total</b>                     | —                                               |
| 11 | Fournisseurs LLM                      | <b>Adaptive Router + contrat d'adaptateur</b> (New API, Ollama, Groq, Cerebras, GPT4Free) | Négociation de coût dynamique                   |
| 12 | Observabilité                         | <b>Logs structurés JSON + OpenTelemetry</b>                                               | Dashboards temps réel                           |
| 13 | Noyau (Core / Manas)                  | <b>Immuable, versionné, modifiable uniquement par l'humain</b>                            | —                                               |

Ces 13 lignes sont l'ancre. Le reste du document les détaille et les justifie (voir chapitre ADR pour le "pourquoi").

## Chapitre 2 — Stack technique

### 2.1 Cœur

- **Python 3.12**, typé (type hints partout, `mypy --strict` en CI).
- **asyncio** comme modèle de concurrence de base.
- **Pydantic v2** pour tous les contrats de données (missions, événements, rapports, capacités). Chaque schéma est versionné.
- **FastAPI** pour l'API de commande/observation.

- **Structlog** pour les logs structurés JSON.

## 2.2 Frontières de modules (packages Python)

```
nexus/  
  core/      # Manas : boucle OODA, décideur unique. IMMUTABLE.  
  router/    # Adaptive Router : cascade 7 niveaux  
  reasoning/ # Formal Reasoning déterministe + Great Sage (conseil)  
  memory/    # Mémoire indexée, consolidation, archivage  
  belzebuth/ # Mission -> fiche de savoir-faire  
  evolution/ # Evolution Engine + Gymnase  
  capabilities/ # Capability Creation Engine + registre  
  fabric/    # Execution Fabric : isolation, budgets, kill-switch  
  guard/     # Mission Guard (pendant) + Validation Engine (après)  
  bus/       # Event Bus (abstraction)  
  api/       # FastAPI  
  storage/   # accès PostgreSQL / pgvector / Redis
```

Règle : `core/` ne dépend d'AUCUN autre package sauf `bus/` (émission d'événements) et des contrats. Tout le reste s'abonne au noyau, jamais l'inverse.

---

## Chapitre 3 — Déploiement (interne d'abord)

### 3.1 Cible V1 : Docker Compose, hôte unique

Services :

- `nexus-core` (API + boucle OODA)
- `postgres` (état + pgvector)
- `redis` (cache sémantique + Event Bus à partir de Phase 2)
- `nexus-fabric-runner` (exécution isolée des capacités)

Pourquoi pas Kubernetes en V1 : coût opérationnel disproportionné pour un usage interne mono-hôte. K8s est prévu comme chemin de sortie (chapitre 13, phase spéculative), pas comme point de départ.

### 3.2 Environnements

- `dev` : SQLite autorisé en fallback pour tests rapides ; tout le reste identique.
- `interne` (cible réelle) : PostgreSQL, Redis, conteneurs.

### 3.3 Configuration

- Un unique fichier `nexus.config.yaml` versionné + surcharges par variables d'environnement.
- Aucune valeur "magique" dans le code : tout seuil vit dans la config (voir chapitre 9).

---

## Chapitre 4 — Données & persistance

### 4.1 Répartition

| Donnée                             | Stockage                                | Justification                         |
|------------------------------------|-----------------------------------------|---------------------------------------|
| Missions, états, itérations        | PostgreSQL                              | transactionnel, requêtable            |
| Fiches de savoir-faire (Belzébuth) | PostgreSQL                              | relationnel + versionné               |
| Mémoire sémantique (embeddings)    | pgvector                                | co-localisé, un seul système à opérer |
| Politiques & preuves Gymnase       | PostgreSQL                              | audit, rejouabilité                   |
| Registre de capacités              | PostgreSQL                              | statut probatoire/promu traçable      |
| Cache sémantique                   | Redis                                   | latence, TTL                          |
| Journaux d'événements              | PostgreSQL (table append-only) + export | audit immuable                        |

## 4.2 Principes

- **Aucune suppression physique** des missions et capacités : archivage logique ( `archived_at` ).
- **Migrations** via Alembic, versionnées, réversibles.
- Le **noyau** persiste sa version ; toute modification du noyau crée une nouvelle version explicite (jamais d'édition en place).

# Chapitre 5 — Event Bus

## 5.1 Décision progressive

- **Phase 1** : `AsyncEventBus` en mémoire (asyncio). Suffit à câbler tout le système et à le tester.
- **Phase 2+** : **Redis Streams**, persistant, avec groupes de consommateurs, sans changer l'interface.

## 5.2 Contrat d'interface (stable dès Phase 1)

```
class EventBus(Protocol):
    async def publish(self, event: Event) -> None: ...
    async def subscribe(self, event_type: str, handler: Handler) -> None: ...
```

Tous les modules dépendent de ce `Protocol` , jamais de l'implémentation. Passer d'in-process à Redis Streams = changer une injection de dépendance, zéro refonte.

## 5.3 Événements canoniques (extrait)

`MissionAccepted` , `MissionPlanned` , `ActionDelegated` , `ActionCompleted` , `MissionGuardTriggered` , `MissionValidated` , `SkillCardCreated` , `CapabilityProposed` , `CapabilityPromoted` , `PolicyProven` , `PolicyRejected` .

Chaque événement porte : `id` , `type` , `schema_version` , `mission_id?` , `timestamp` , `payload` , `causation_id` , `correlation_id` .

---

## Chapitre 6 — API (contrat externe minimal)

Base FastAPI. L'API est volontairement mince : Nexus n'est pas piloté par requêtes fines, il reçoit des missions et rend des rapports.

### 6.1 Endpoints V1

```
POST /mission          -> 202 MissionAccepted { mission_id }
GET  /mission/{id}     -> état + itérations + statut Guard
GET  /mission/{id}/report -> rapport final (voir ch. 11)
POST /mission/{id}/abort -> déclenche kill-switch
GET  /capabilities     -> registre (probatoire/promu)
GET  /health           -> liveness/readiness
GET  /events?mission_id= -> flux d'événements (SSE)
```

### 6.2 Contrat POST /mission

Entrée : `Mission v1` (voir ch. 10). Sortie immédiate : `202` + `mission_id`. Le traitement est **asynchrone** ; le client suit via `/mission/{id}` ou le flux SSE. Aucune exécution synchrone bloquante.

---

## Chapitre 7 — Contrat des fournisseurs (Adaptive Router)

### 7.1 Adaptateur unique

Chaque fournisseur implémente le même contrat :

```
class ProviderAdapter(Protocol):
    name: str
    async def complete(self, req: CompletionRequest) -> CompletionResult: ...
    async def health(self) -> ProviderHealth: ...
    def cost_estimate(self, req: CompletionRequest) -> Cost: ...
    def capabilities(self) -> ProviderCaps: ... # context window, tools, json mode
```

Fournisseurs V1 : **New API** (agrégateur), **Ollama** (local), **Groq**, **Cerebras**, **GPT4Free** (best-effort, jamais sur chemin critique).

### 7.2 Cascade du Router (7 niveaux, rappel V2.5.2)

1. Cache sémantique (Redis) → 2. Modèle local trivial (Ollama) → 3. Modèle rapide low-cost (Groq/Cerebras) → 4. Modèle standard → 5. Modèle fort → 6. Raisonnement profond → 7. Great Sage (conseil, décisions critiques uniquement).

Le Router **descend** la cascade tant que la confiance/contrainte l'exige. Chaque montée de niveau est journalisée avec sa justification (coût vs enjeu).

### 7.3 Déclaration

Les fournisseurs sont déclarés dans `nexus.config.yaml`, jamais en dur. Un fournisseur indisponible est retiré du routage à chaud via `health()`.

---

## Chapitre 8 — Sandboxes & Execution Fabric

## 8.1 Trois niveaux d'isolation (Execution Fabric)

1. **In-process** : capacités pures, sans effet de bord, rapides.
2. **Conteneur** : effets de bord contrôlés, réseau restreint, budgets durs.
3. **Conteneur sans réseau + FS éphémère** : exécution non fiable / capacités probatoires.

Tous portent : budget CPU/mémoire/temps **dur**, `kill-switch`, retries **bornés**, télémétrie.

## 8.2 Sandbox de Capability Creation (règle stricte V2.5.2)

- Aucune capacité n'entre sans **oracle réel préalable** (un jeu de vérité).
- Revue par un **second modèle** distinct du générateur.
- Test en **sandbox niveau 3** (sans réseau, FS temporaire).
- Intégration en statut **probatoire**.
- Promotion seulement après **K succès réels** (voir ch. 9).
- Rollback total et instantané : la sandbox ne laisse aucun résidu.

---

# Chapitre 9 — Valeurs par défaut (enfin écrites)

Toutes surchargeables en config. Ce sont les valeurs de départ, choisies prudemment pour un usage interne.

## Mission Guard

```
max_iterations      = 25
max_cost_usd        = 5.00
timeout_seconds     = 900
max_parallelism     = 4
max_tool_calls      = 100
```

## Execution Fabric (par action)

```
cpu_limit           = 1 core
memory_limit_mb     = 1024
action_timeout_s    = 120
max_retries         = 2
```

## Capability Creation

```
oracle_min_cases    = 5
probation_min_success = 10      # K : succès réels avant promotion
probation_max_failure = 0       # tout échec réel repousse la promotion
review_model_distinct = true
```

## Evolution Engine / Gymnase

```
gym_replay_min_runs = 20
policy_promote_delta = +5%      # amélioration prouvée minimale
policy_regression_veto = any     # une régression = rejet
```

## Router

```
cache_ttl_seconds      = 3600
confidence_threshold    = 0.75    # sous ce seuil, on monte d'un niveau
great_sage_trigger      = critical_only
```

## Chapitre 10 — Schémas JSON versionnés

### 10.1 Règle

Tout objet inter-modules porte `schema_version`. Une montée de version s'accompagne d'un **migrateur** `vN -> vN+1` testé.

### 10.2 Mission v1

```
{
  "schema_version": 1,
  "mission_id": "uuid",
  "objective": "string",
  "constraints": { "deadline": null, "budget_usd": 5.0, "forbidden": [] },
  "context": { "inputs": [], "references": [] },
  "priority": "normal",
  "created_by": "human|nexus",
  "created_at": "iso-8601"
}
```

### 10.3 Migration Mission v1 -> v2 (exemple de discipline)

`v2` ajoute `success_criteria[]`. Migrateur : si absent, dériver depuis `objective` + marquer `criteria_inferred=true`.  
Aucun champ jamais supprimé sans migrateur.

### 10.4 SkillCard, Capability, Report

Même discipline : `schema_version` obligatoire, migrateurs testés, rétrocompat lecture.

## Chapitre 11 — Rapports

Un rapport final de mission ( `Report v1` ) contient :

```
{
  "schema_version": 1,
  "mission_id": "uuid",
  "verdict": "PASS|FAIL|PARTIAL",
  "objective": "...",
  "summary": "...",
  "iterations": 7,
  "cost_usd": 1.83,
  "duration_s": 412,
  "actions": [ { "type": "...", "result": "...", "isolation": "..." } ],
  "validation": { "criteria": [ { "name": "...", "passed": true } ] },
  "learned": { "skill_cards": [], "capabilities_touched": [] },
  "guard_events": [],
  "generated_at": "iso-8601"
}
```

Le rapport est produit par le **Validation Engine** (jugement APRÈS), distinct du Mission Guard (interruption PENDANT).

---

## Chapitre 12 — Sécurité

Pas seulement Red Team. Quatre surfaces protégées :

1. **Noyau** : immuable au runtime, modifiable uniquement par l'humain via redéploiement versionné. Aucun module ne peut écrire dans `core/`.
2. **Mémoire** : écritures via API interne validée ; pas d'injection directe ; archivage append-only pour l'audit.
3. **Capacités** : aucune promotion sans le pipeline du ch. 8 ; capacité probatoire toujours en isolation niveau 3.
4. **APIs / fournisseurs** : secrets hors code (variables d'env / coffre) ; GPT4Free jamais sur chemin critique ; budgets durs empêchent l'emballlement de coût.

Kill-switch global : `POST /mission/{id}/abort` + arrêt d'urgence propageant un `MissionGuardTriggered` de niveau `hard_stop`.

---

## Chapitre 13 — Plan de construction par phases

Règle absolue : à la fin de CHAQUE phase, Nexus démarre, reçoit une mission, et rend un rapport. Chaque phase finit par un verdict **PASS/FAIL** mesurable.

### Phase 0 — Squelette vivant

**Livable** : `POST /mission` accepte une mission, le noyau exécute une boucle OODA triviale (un seul fournisseur local via Ollama), rend un `Report v1`. Bus in-process. PostgreSQL + migrations.

**PASS si** : une mission triviale entre et ressort avec un rapport `PASS`, tout journalisé.

### Phase 1 — Router en cascade réel

**Livable** : Adaptive Router niveaux 1-5, cache Redis,  $\geq 2$  fournisseurs, journalisation des montées de niveau.

**PASS si** : même mission résolue au niveau le moins coûteux suffisant ; montée de niveau tracée et justifiée.

### Phase 2 — Mémoire + Belzébuth

**Livable** : mémoire indexée (pgvector), consolidation, et Belzébuth qui transforme une mission réussie en `SkillCard`.

**PASS si** : une mission similaire ultérieure réutilise une fiche de savoir-faire mesurablement (moins d'itérations/coût).

### Phase 3 — Execution Fabric + Mission Guard + Validation Engine

**Livable** : 3 niveaux d'isolation, budgets durs, kill-switch, Guard pendant / Validation après, rapports complets.

**PASS si** : une mission qui dépasse un budget est interrompue proprement et rapportée `FAIL` avec cause.

### Phase 4 — Gymnase + Evolution Engine

**Livable** : Gymnase rejouable, preuves de politiques, promotion seulement sur amélioration prouvée sans régression.

**PASS si** : une politique n'est promue qu'après  $\geq 20$  replays et +5% sans régression ; une régression est vetoée.

### Phase 5 — Capability Creation Engine



**Livable** : pipeline complet (oracle → revue second modèle → sandbox niveau 3 → probatoire → promotion après K succès).

**PASS si** : aucune capacité ne peut être promue en contournant une seule étape ; rollback prouvé.

## Phase 6 — Great Sage + Formal Reasoning

**Livable** : conseil sur décisions critiques uniquement, raisonnement déterministe pour contraintes logiques.

**PASS si** : Great Sage n'est sollicité que sur **critical**, et le raisonnement formel produit des résultats reproductibles.

## Phase 7 — Durcissement (Event Bus persistant + sécurité + observabilité)

**Livable** : bascule vers Redis Streams, OpenTelemetry, revue sécurité des 4 surfaces.

**PASS si** : redémarrage sans perte d'événements ; traces bout-en-bout d'une mission.

## Phases spéculatives (V1.5+ / non bloquantes)

- Kubernetes / multi-nœuds.
- microVM (Firecracker) pour isolation renforcée.
- Neo4j si le raisonnement graphe devient central.
- **Fusion / autonomie avancée** : traité en dernier, jamais avant que les phases 0-7 soient toutes PASS.

---

# Chapitre 14 — ADR : Décisions d'Architecture figées

Format : chaque décision documente **Décision / Pourquoi / Conséquences / Alternatives rejetées**. Quand, dans six mois, on demandera « pourquoi ce choix ? », la réponse est ici.

## D001 — Le noyau Core est immuable au runtime

- **Décision** : **core/** (Manas, boucle OODA) n'est jamais modifié par Nexus lui-même. Modification uniquement par l'humain, via redéploiement versionné.
- **Pourquoi** : garantir que le décideur reste prévisible et auditable ; l'évolution vit dans l'espace évolutif, pas dans le décideur.
- **Conséquences** : toute "amélioration du noyau" passe par un cycle humain explicite ; plus lent mais sûr.
- **Alternatives rejetées** : noyau auto-modifiable (risque d'emballement, perte d'auditabilité, impossible à raisonner formellement).

## D002 — Le Router utilise une cascade, pas une classification directe

- **Décision** : routage économique en 7 niveaux, montée progressive.
- **Pourquoi** : minimise le coût en résolvant au niveau le moins cher suffisant ; dégrade proprement.
- **Conséquences** : légère latence de montée ; journalisation nécessaire pour comprendre les choix.
- **Alternatives rejetées** : classifieur unique décidant a priori du modèle (fragile, coûteux à entraîner, opaque, mauvaise dégradation).

## D003 — Belzébuth ne modifie jamais directement une capacité

- **Décision** : Belzébuth produit des **fiches de savoir-faire**, il ne touche pas au registre de capacités.

- **Pourquoi** : séparer l'apprentissage descriptif (ce qui a marché) de la création exécutable (ce qui devient une capacité). La création passe par un pipeline sûr.
- **Conséquences** : deux artefacts distincts (SkillCard vs Capability) ; traçabilité claire.
- **Alternatives rejetées** : Belzébuth écrivant directement des capacités (contourne oracle/sandbox/probation = faille de sécurité majeure).

## D004 — Capability Creation passe toujours par le Gymnase / la sandbox

- **Décision** : aucune capacité promue sans oracle → revue second modèle → sandbox niveau 3 → probation → K succès réels.
- **Pourquoi** : empêcher qu'une capacité non prouvée agisse sur le système réel.
- **Conséquences** : création plus lente ; garantie forte de non-régression.
- **Alternatives rejetées** : promotion sur simple succès unique ou sur jugement d'un seul modèle (biais, faux positifs).

## D005 — pgvector plutôt qu'un moteur vectoriel dédié en V1

- **Décision** : embeddings dans PostgreSQL via pgvector.
- **Pourquoi** : un seul système à opérer en interne ; transactions cohérentes état+mémoire.
- **Conséquences** : plafond de performance ; migration vers Chroma/FAISS possible si la volumétrie explose.
- **Alternatives rejetées** : Chroma/FAISS dès le départ (deuxième système à opérer, complexité prématurée pour un usage interne).

## D006 — Event Bus in-process d'abord, Redis Streams ensuite, derrière une même interface

- **Décision** : `EventBus` est un `Protocol` ; implémentation in-process en Phase 1, Redis Streams en Phase 7.
- **Pourquoi** : livrer un système fonctionnel vite, sans se lier à une infra lourde ; migrer sans refonte.
- **Conséquences** : discipline d'interface stricte dès le début.
- **Alternatives rejetées** : Kafka dès le départ (surdimensionné pour interne mono-hôte).

# Chapitre 15 — Ce qui reste explicitement HORS V3

- Distribution externe / multi-tenant.
  - Connexion GitHub et pipeline de build réel : **à activer au moment de la construction**, pas maintenant.
  - Toute nouvelle capacité cognitive non décrite en V2.5.2.
- Verrou** : à la clôture de cette V3, on ne rouvre plus la conception fondamentale. On ouvre un dépôt, on construit la Phase 0, et on valide phase par phase.